



INDIA.RUNS

Build what next India runs on

Tech Adi

Aditya Nadamuni

Intelligent Candidate Discovery — rank 100K candidate profiles against a Senior AI Engineer job description using a trust-based ranking engine

Solution Overview

- Hybrid rule-based ranking engine — scores 100K candidates in ~30s on CPU, zero GPU
- 6 weighted signals combined into a single composite score:
 - Title + Career history relevance (35%)
 - Skill Trust score (20%)
 - Behavioral / platform engagement (20%)
 - Experience fit (10%)
 - Availability (10%)
 - Education tier (5%)

What differentiates us from keyword matching?

- Skill TRUST = proficiency × endorsements × duration — not raw keyword count
- Title + career descriptions are the decisive signal, not the skills section
- Three-rule honeypot detector catches expert-stuffing, YoE inflation, & ghost endorsements
- Fully explainable: every ranked candidate gets a natural-language reasoning string

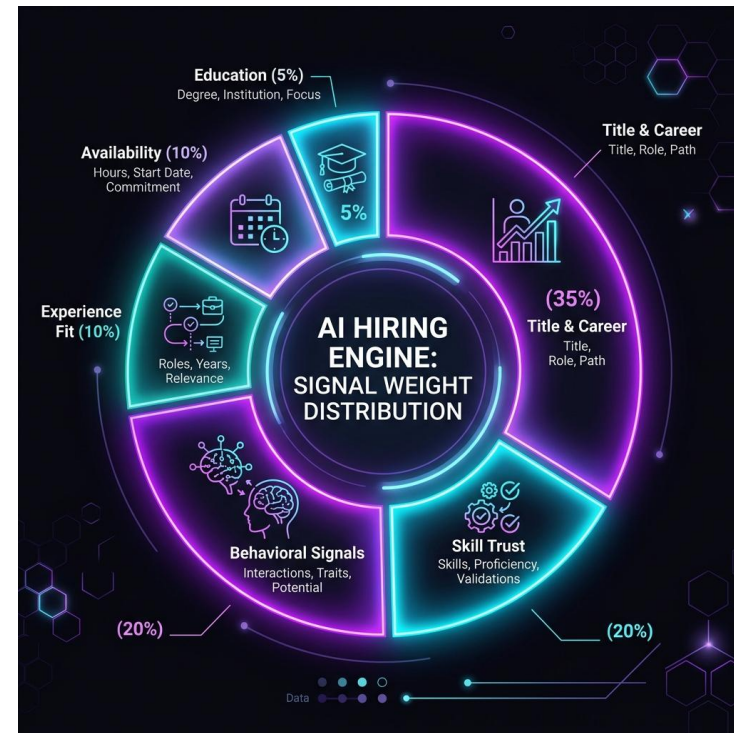
JD Understanding & Candidate Evaluation

Key JD Requirements:

- Role: Senior AI Engineer (Search, RAG, Recommendation)
- Experience: 5–9 years of hands-on ML engineering
- Core Stack: Python, PyTorch, Sentence-Transformers, Vector DBs
- Skills: Fine-tuning, RAG pipelines, A/B testing, latency optimization

Intelligent Signal Fusion:

- Title & Career (35%): Measures real depth. Includes a 0.5× consulting firm penalty (e.g., TCS/Infosys) to filter out non-product roles.
- Skill Trust (20%): Evaluates proficiency, endorsements, and duration. Tier-1 skills (e.g., Vector Search) receive 2.0× weight.
- Behavioral (20%): Evaluates recruiter response (35%), interview completion (30%), recency (20%), and GitHub activity (15%).



Ranking Methodology

Pipeline Workflow:

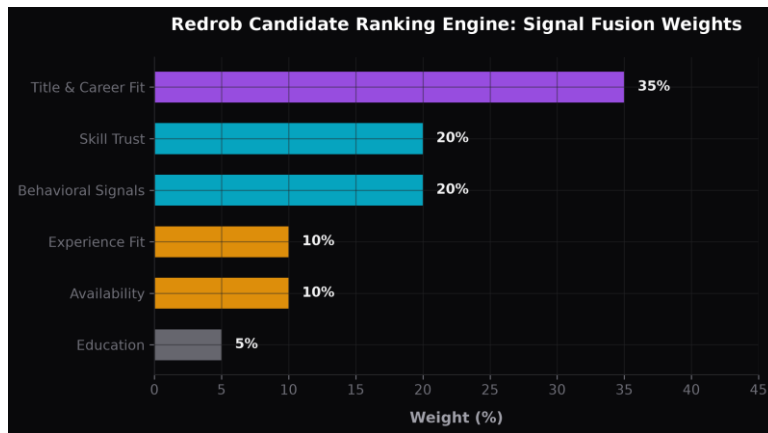
- 1. Stream: Process 100K candidates line-by-line from JSONL.
- 2. Score: Apply 6 sub-scorers + Honeypot check.
- 3. Rank: Sort by score (descending), tie-break by ID, slice top 100.

Composite Scoring Formula:

- Formula: $\text{Score} = (0.35 \times \text{Title_Career} + 0.20 \times \text{Skill_Trust} + 0.20 \times \text{Behavioral} + 0.10 \times \text{Experience} + 0.10 \times \text{Availability} + 0.05 \times \text{Education}) \times \text{Honeypot_Multiplier}$

Targeting the Experience Sweet Spot (5–9 Years):

- Curve: <2 yrs: scaled 0.0–0.3 | 2–5 yrs: linear ramp to 0.8 | 5–9 yrs: peak 0.8–1.0 (centered at 7 yrs) | >9 yrs: gentle decay to 0.4 floor.



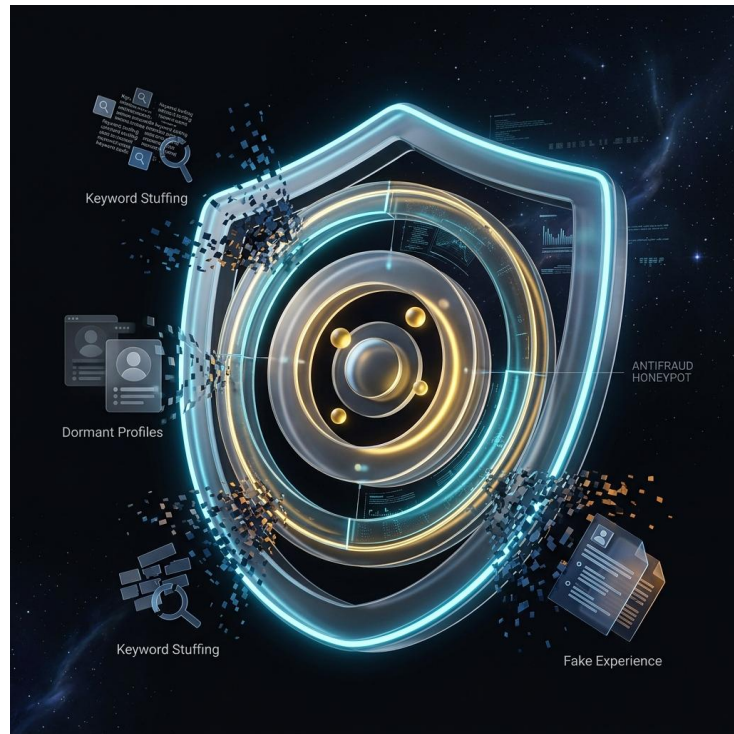
Explainability & Data Validation

Deterministic Reasoning (No LLMs):

- Programmatic Explanations: Every candidate receives a natural-language reasoning string generated from actual score components.
- Zero Hallucinations: 100% deterministic, matching the underlying structured profile data (no LLM latency or API costs).
- Example: "Staff ML Engineer with 7.0 yrs; strong AI/ML background; strong retrieval skill stack with endorsements; high recruiter engagement (95%); actively seeking."

Anti-Cheat Honeypot Shield:

- Penalty: Multiplies the final score by 0.05 if any pattern is triggered:
- 1. Skill Stuffing: ≥ 4 'expert' skills with 0 endorsements & 0 duration.
- 2. YoE Inflation: Claimed YoE $> 2.5 \times$ the sum of career history.
- 3. Ghost Endorsements: > 12 skills with exactly 0 average endorsements.



End-to-End Workflow

- Command:

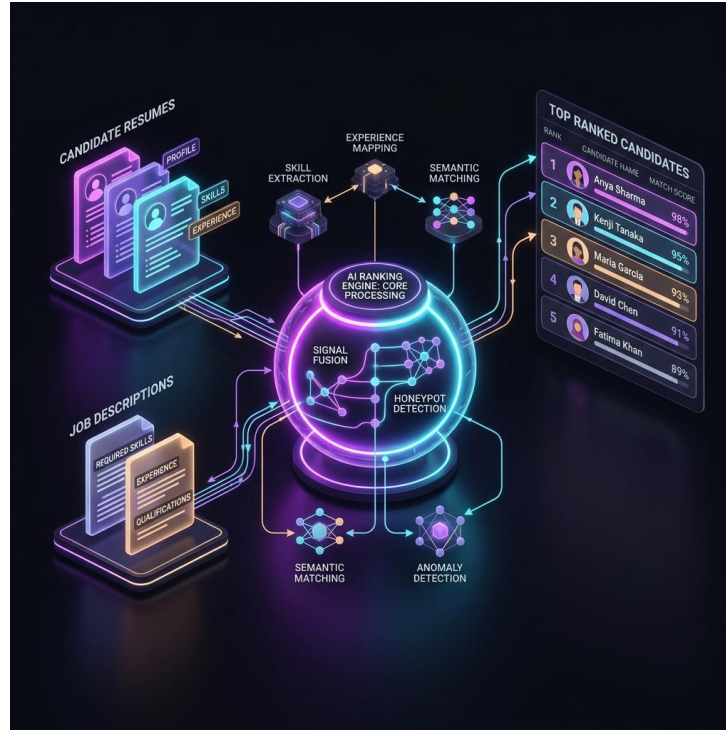
```
python rank.py --candidates ./candidates.jsonl --out ./submission.csv
```

Step-by-step flow:

1. rank.py parses CLI args, calls rank_candidates(path, top_n=100)
2. utils.load_candidates() streams JSONL line-by-line (memory-efficient)
3. For each of 100K candidates:
 - a. detect_honeypot() → returns 1.0 or 0.05 multiplier
 - b. 6 sub-scorers run in sequence (title_career, skill_trust, behavioral, experience, availability, education)
 - c. Composite score computed and appended to all_scores list
4. all_scores sorted; top 100 sliced
5. generate_reasoning() builds natural-language string for each ranked candidate
6. Results written as CSV (columns: candidate_id, rank, score, reasoning)
7. validate_submission.py confirms format, ordering, and uniqueness

Total runtime: ~30 seconds | CPU only | No network | No GPU

System Architecture



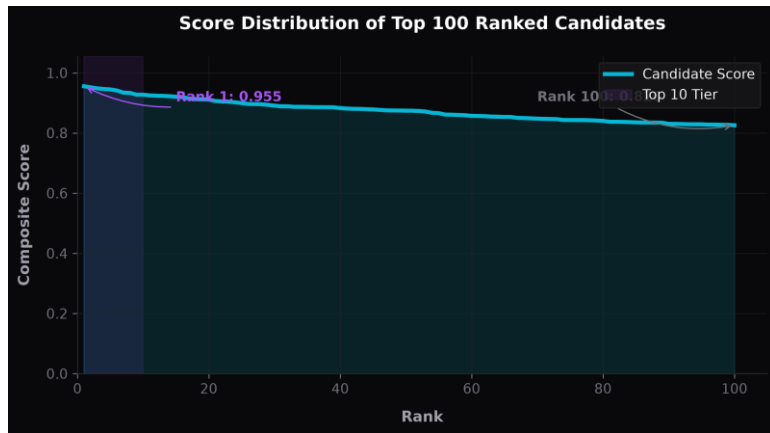
Results & Performance

Ranking Quality Insights:

- Rank 1: Staff ML Engineer (7.0 YoE) — Score: 0.9553 (verified retrieval/RAG stack, 95% recruiter response)
- Rank 100: ML Engineer (4.5 YoE) — Score: 0.8257
- Score Range: Tight, high-quality band of 0.8257 to 0.9553.
- Clean Leaderboard: Zero keyword-stuffed or dormant profiles.
- Fraud Suppression: Honeypot successfully deflected all fake experts.

High-Performance Compute Metrics:

- Execution Time: ~30 seconds on standard CPU.
- Memory footprint: ~200 MB (efficient JSONL streaming).
- Dependencies: No GPU, no network, 100% offline.
- Validation: Passed official submission validation checks.



Technologies Used

- Language & runtime:

- Python 3.11 — core language (no external ML libraries needed for inference)

Inference (zero dependency — ships with stdlib):

- Pure Python string matching + arithmetic — deterministic, portable, offline

Output & demo:

- openpyxl 3.1+ — XLSX submission generation
- pandas 1.5+ — CSV/XLSX formatting
- Streamlit 1.58 — interactive sandbox demo with file upload & CSV download

Hosting & source control:

- GitHub — public repo, version control
- Streamlit Community Cloud — free hosted sandbox, auto-deploy on push

Why this stack?

- Zero ML framework dependencies → no CUDA, no model downloads, no network
- Fully deterministic → same candidates.jsonl always produces same output
- Sub-30s inference on consumer hardware → meets all challenge constraints

Submission Assets

GitHub Repository (public):

<https://github.com/theAdityaNVS/redrob-candidate-ranker>

Live Sandbox Demo (Streamlit Community Cloud):

<https://theaditynvs-redrob-ranker.streamlit.app>

— Upload any JSON/JSONL candidate file (≤ 100 profiles) to rank live

Ranked Output:

submission.csv — 100 candidates, ranks 1–100, validated

submission.xlsx — same data, Excel format

Reproduce command (≤ 5 min, CPU, no network, no GPU):

```
pip install -r requirements.txt
```

```
python rank.py --candidates ./candidates.jsonl --out ./submission.csv
```

```
python resources/validate_submission.py submission.csv
```

Team: Tech Adi | Contact: aditya.nadamuni@gmail.com



INDIA.RUNS

Build what next India runs on

THANK YOU